

		semantik menggunakan LLM.	input secara menyeluruh (sintaksis & semantik) pada berbagai jenis elemen input.	potensi yang diidentifikasi oleh penelitian-penelitian sebelumnya, khususnya [5] dan [6].
--	--	---------------------------	--	---

Dari Tabel 2.1, terlihat bahwa terdapat kebutuhan yang jelas akan metode validasi input yang lebih canggih, khususnya yang mampu menangani aspek semantik. Penelitian sebelumnya telah menunjukkan keterbatasan validasi sintaksis murni dan potensi LLM dalam pemahaman semantik serta aplikasinya dalam validasi semantik input. Penelitian ini bertujuan untuk mengisi celah dengan mengembangkan sebuah *library* validasi *front-end* yang praktis, yang mengintegrasikan kemampuan LLM untuk melakukan validasi semantik, sebagai pengembangan dari fokus validasi sintaksis yang dominan saat ini dan mengadopsi temuan dari Shin dkk. [5] dan Alian dkk. [6] untuk menghasilkan alat bantu bagi *developer*.

2.2. Landasan Teori

2.2.1. Validasi Input

Validasi input adalah proses untuk memastikan bahwa data yang dimasukkan oleh pengguna ke dalam suatu sistem adalah akurat, konsisten, dan sesuai dengan aturan atau kriteria yang telah ditetapkan sebelum data tersebut diproses atau disimpan. Tujuan utama validasi input adalah untuk menjaga integritas data, meningkatkan keamanan sistem dengan mencegah input berbahaya (misalnya, SQL Injection, Cross-Site Scripting), dan memberikan pengalaman pengguna yang lebih baik dengan memberikan umpan balik yang jelas mengenai kesalahan input.

Contoh validasi input dapat di lihat pada gambar 2.1 berikut .

```

public function validateEmail($attribute, $value, $parameters)
{
    if (! is_string($value) && ! (is_object($value) && method_exists($value, '__toString'))) {
        return false;
    }

    $validations = (new Collection($parameters))
        ->unique()
        ->map(fn ($validation) => match (true) {
            $validation === 'strict' => new NoRFCWarningsValidation(),
            $validation === 'dns' => new DNSCheckValidation(),
            $validation === 'spooof' => new SpooofCheckValidation(),
            $validation === 'filter' => new FilterEmailValidation(),
            $validation === 'filter_unicode' => FilterEmailValidation::unicode(),
            is_string($validation) && class_exists($validation) => $this->container->make($validation),
            default => new RFCValidation(),
        })
        ->values()
        ->all() ?: [new RFCValidation()];

    $emailValidator = Container::getInstance()->make(EmailValidator::class);

    return $emailValidator->isValid($value, new MultipleValidationWithAnd($validations));
}

```

Gambar 2.1. Contoh validasi input.

Pada gambar 2.1 merupakan fungsi `validateEmail` dari salah satu framework yang banyak digunakan yaitu laravel, fungsi ini memvalidasi alamat email dengan mendukung beberapa mode validasi tambahan seperti `strict`, `dns`, `spooof`, dan `filter`. Pertama, fungsi memastikan bahwa nilai yang divalidasi bisa dikonversi ke string. Kemudian, ia membentuk koleksi aturan validasi berdasarkan parameter yang diberikan, mengubahnya menjadi objek validasi sesuai jenisnya. Misalnya, `DNSCheckValidation` untuk pengecekan DNS dan `SpooofCheckValidation` untuk deteksi email palsu. Jika tidak ada parameter, default-nya menggunakan `RFCValidation`. Terakhir, fungsi menggunakan instance `EmailValidator` untuk mengecek apakah email valid dengan menggabungkan semua aturan tersebut menggunakan `MultipleValidationWithAnd`.

2.2.2. Validasi Sintaksis

Validasi sintaksis adalah jenis validasi input yang berfokus pada pemeriksaan format, struktur, atau pola teknis dari data input tanpa memperhatikan makna atau konteksnya. Contohnya termasuk memeriksa apakah input email memiliki format "nama@domain.com", apakah input tanggal sesuai format "YYYY-MM-DD", atau apakah input nomor telepon

hanya berisi digit numerik dengan panjang tertentu. Meskipun efektif untuk memastikan konsistensi format data, validasi sintaksis tidak cukup untuk menangkap semua jenis kesalahan, terutama yang berkaitan dengan kelayakan semantik.

Contoh validasi sintaksis dapat di lihat pada kode program 2.1 berikut.

```

1. Function validateEmail(email):
2. pattern = "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-
   ]+\\. [a-zA-Z]{2,}$"
3. If match(email, pattern):
   Return "Email valid secara
   sintaksis."
4. Else:
   Return "Email tidak valid secara
   sintaksis."
5. Function validateAge(ageInput):
6. pattern = "^(1[01][0-9]|120|[1-9][0-9]?)$"
   // Valid: 1-120
7. If match(ageInput, pattern):
   Return "Umur valid secara sintaksis."
8. Else:
   Return "Umur tidak valid secara
   sintaksis."

```

Kode Program 2.1 Contoh validasi secara sintaksis.

2.2.3. Validasi Semantik

Validasi semantik melangkah lebih jauh dari validasi sintaksis dengan memeriksa makna, konteks, dan kelogisan dari data input. Tujuannya adalah untuk memastikan bahwa input tidak hanya benar secara format, tetapi juga masuk akal dalam konteks aplikasi atau domain tertentu. Sebagai contoh, validasi semantik dapat memeriksa apakah usia yang dimasukkan (misalnya, "-5" atau "250") adalah usia yang valid untuk manusia, atau apakah sebuah nama produk yang dimasukkan tidak mengandung kata-kata yang tidak pantas, meskipun secara sintaksis (sebagai string) input tersebut valid. Myllyaho dkk. [2], menekankan bahwa pendekatan validasi semantik masih kurang diadopsi secara luas. Berikut contoh validasi semantik lainnya pada tabel 2.2.

Tabel 2.2 Contoh Validasi Semantik

Jenis Input	Contoh Valid	Contoh Tidak Valid	Alasan Tidak Valid
-------------	--------------	--------------------	--------------------

Nama Orang	Andri Pratama	@ndri123	Mengandung simbol dan angka, tidak lazim untuk nama orang
Alamat / Nama Jalan	Jl. Diponegoro No. 12	1234567890 panjang banget jalan gila	Tidak natural, terlalu panjang, tidak bermakna sebagai alamat
Usia	1 - 80	-5 atau 1000	Usia seseorang tidak mungkin minus atau pun hingga 1000 tahun
Judul	Pengaruh Media Sosial terhadap Remaja	!!! atau a	Terlalu pendek atau hanya simbol, tidak menjelaskan apapun
Pekerjaan	Software Engineer	Ngupil Profesional	Tidak umum atau tidak logis sebagai profesi resmi
Nama Perusahaan	PT Sumber Rejeki Abadi	asdf atau Sex Shop	Nama tidak bermakna atau mengandung konten tidak pantas
Nama Institusi	Universitas Gadjah Mada	Toko Bunga Mawar	Tidak cocok sebagai institusi pendidikan atau resmi
Nama Produk	Sabun Cuci Muka GlowUp	Obat Kuat Lelaki Tahan Lama	Produk mengandung klaim sensitif / tidak sesuai etika domain umum
Nama Lokasi	Jakarta Selatan	Negeri Dongeng Pelangi Ke-13	Nama fiktif, tidak bisa diverifikasi sebagai lokasi nyata
Tag	teknologi, pemrograman, edukasi	!!!, teknologi, teknologi, asdf	Mengandung simbol, duplikat, atau tag tak bermakna
Deskripsi (textarea)	Aplikasi ini membantu pengguna mengatur keuangan harian mereka dengan fitur pengingat otomatis dan laporan grafik.	, a, atau test test test	Tidak informatif, terlalu pendek, tidak mengandung makna nyata
Cerita	Hari ini saya belajar membuat program Python pertama saya. Awalnya bingung, tapi akhirnya bisa.	👍👍👍, Cerita	Terlalu pendek atau hanya simbol, tidak memenuhi struktur naratif
Komentar	Sangat membantu! Terima kasih.	Bodoh banget lu!, asdf, 🙄🙄🙄	Komentar kasar, spam, atau hanya emoji

2.2.4. API Publik

API Publik adalah antarmuka pemrograman yang secara eksplisit diekspos oleh sebuah modul, pustaka, atau layanan perangkat lunak agar dapat diakses dan digunakan oleh entitas eksternal. Berbeda dengan logika internal (internal implementation) yang bersifat tertutup, API publik bertindak sebagai kontrak interaksi antara penyedia layanan (library) dan pengguna layanan (client). Kontrak ini mendefinisikan fungsi, metode, kelas, atau struktur data apa saja yang tersedia untuk dipanggil, beserta aturan input dan output yang harus dipatuhi.

Prinsip utama dari API publik adalah enkapsulasi atau menyembunyikan informasi (information hiding). Dengan memisahkan antarmuka publik dari implementasi privat, pengembang library dapat mengubah, memperbaiki, atau mengoptimalkan kode di balik layar tanpa mengganggu aplikasi lain yang bergantung pada library tersebut, selama struktur API publiknya tidak berubah. Dalam bahasa pemrograman modern seperti Rust, batasan ini ditegakkan secara ketat melalui pengubah akses (access modifiers), di mana hanya komponen yang diberi tanda pub (public) yang dapat diakses oleh modul luar. Kualitas API publik yang baik ditandai dengan kemudahan penggunaan (usability), dokumentasi yang jelas, dan konsistensi perilaku.

2.2.5. Library (Perpustakaan Perangkat Lunak)

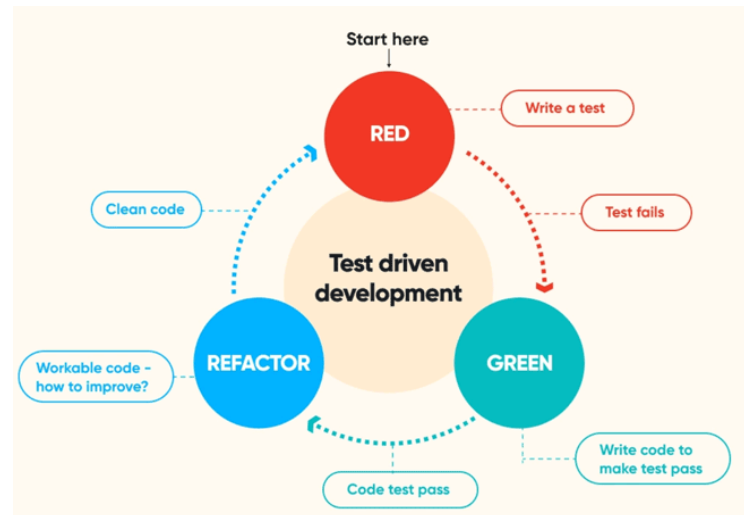
Dalam pengembangan perangkat lunak, *library* (perpustakaan) adalah kumpulan kode yang telah ditulis sebelumnya yang menyediakan fungsionalitas tertentu sehingga dapat digunakan kembali oleh pengembang dalam berbagai proyek. Penggunaan *library* bertujuan untuk mempercepat proses pengembangan, mengurangi redundansi kode, dan meningkatkan modularitas aplikasi. *Library* yang akan dikembangkan dalam penelitian ini bertujuan untuk menyediakan fungsionalitas validasi input semantik yang mudah diintegrasikan ke dalam aplikasi front-end.

2.2.6. Metode Test-Driven Development (TDD)

Metode *Test-Driven Development* (TDD) menjadi pendekatan yang sangat fundamental dalam pengembangan *library* pemrograman, karena ia memfokuskan proses pada pembuatan *Application Programming Interface* (API) yang andal dan teruji. Dalam konteks *library*, TDD mewajibkan pengembang untuk menulis *unit test* yang menggunakan API publik terlebih dahulu, sebelum mengimplementasikan logika internal dari API tersebut. Pendekatan ini secara efektif menjadikan tes sebagai spesifikasi hidup (*living specification*) dan pengguna pertama dari API, sehingga memastikan bahwa setiap fungsionalitas yang ditawarkan kepada developer lain memiliki kontrak yang jelas dan terukur sejak awal [7] .

Proses ini berjalan dalam siklus pendek dan berulang yang dikenal sebagai siklus *Red-Green-Refactor*, yang berfokus pada validasi kontrak API:

1. Tahap Red (Merah): Pengembang mendefinisikan sebuah skenario penggunaan API (misalnya, memanggil sebuah fungsi dengan parameter tertentu) dan menulis tes yang memanggil API tersebut sesuai skenario. Karena implementasi internalnya belum ada, tes ini dipastikan akan gagal.
2. Tahap Green (Hijau): Pengembang menulis implementasi internal seminimal mungkin, cukup untuk membuat "kontrak" dari skenario tes tersebut terpenuhi dan tes menjadi berhasil.
3. Tahap Refactor (Refaktor): Di tahap ini, fokus utama adalah pada perbaikan dan optimalisasi kode internal *library* tanpa mengubah *signature* (nama, parameter, dan tipe kembalian) atau perilaku API publik yang sudah diuji. Keberadaan rangkaian tes yang sudah ada berfungsi sebagai jaring pengaman (*safety net*) yang sangat krusial, memastikan bahwa perbaikan internal tidak merusak fungsionalitas yang telah dijanjikan kepada pengguna *library* [8]. Alur metode TDD secara umum dapat dilihat pada Gambar 2.2.



Gambar 2.2. Siklus Metode *Test-Driven Development* (TDD)[9]

2.2.7. Pengujian Fungsionalitas

Pengujian fungsional merupakan pilar utama dalam penjaminan kualitas sebuah *library* pemrograman, yang bertujuan untuk memverifikasi bahwa setiap *Application Programming Interface* (API) publik yang diekspos telah berfungsi sesuai dengan spesifikasi dan kontrak API yang didokumentasikan. Pengujian ini dilakukan dengan cara memanggil fungsi-fungsi publik dengan berbagai macam argumen masukan dan memvalidasi bahwa *return value* (nilai kembalian) serta *side effects* (seperti pembuatan file atau perubahan *state*) yang dihasilkan sudah benar, tanpa perlu memperhatikan detail logika internalnya. Sebagaimana ditekankan oleh Ammann dan Offutt [10], pengujian sistem (*system testing*) dirancang untuk menentukan apakah sistem yang telah disusun memenuhi spesifikasinya.

Salah satu pendekatan fundamental dalam pengujian fungsional sebuah *library* adalah *black box testing*. Metode ini menguji fungsionalitas *library* dari perspektif *developer* pengguna, di mana pengujian dilakukan dengan berinteraksi hanya melalui API publik tanpa memiliki pengetahuan tentang struktur kode internalnya. Myers *dkk* [11], menjelaskan bahwa metode ini sangat efektif untuk memvalidasi apakah sebuah komponen perangkat lunak (seperti *library*) memenuhi kontrak yang dijanjikan. Dengan menyusun skenario

pengujian yang mencakup berbagai input valid, tidak valid, dan kasus tepi (*edge cases*), *black box testing* pada sebuah *library* dapat secara efisien mengidentifikasi kesalahan pada fungsi API, validasi parameter, penanganan *error*, serta memastikan bahwa perilaku *library* konsisten dan sesuai dengan dokumentasinya. Pendekatan ini sering diimplementasikan dalam bentuk tes integrasi, yang mensimulasikan penggunaan nyata *library* oleh aplikasi lain.

2.2.8. Large language model (LLM)

2.2.8.1. Landasan Teori Large language model (LLM)

Secara fundamental, LLM adalah model probabilistik. Tujuan utamanya adalah untuk menghitung probabilitas urutan kata (atau "token"). Dengan kata lain, jika diberikan urutan kata-kata tertentu, LLM akan memprediksi kata apa yang paling mungkin muncul selanjutnya. Kemampuan inilah yang menjadi dasar dari semua kemampuannya yang lain, mulai dari menjawab pertanyaan hingga menulis puisi.

2.2.8.2. Cara Kerja LLM: Dari Teks ke Pemahaman

Proses kerja LLM dapat dipecah menjadi beberapa tahapan utama, yang hampir semuanya berpusat pada arsitektur revolusioner yang disebut Transformer. Berikut adalah tahapan-tahapan utama dalam cara kerja LLM tersebut:

1. Tokenisasi (Tokenization)

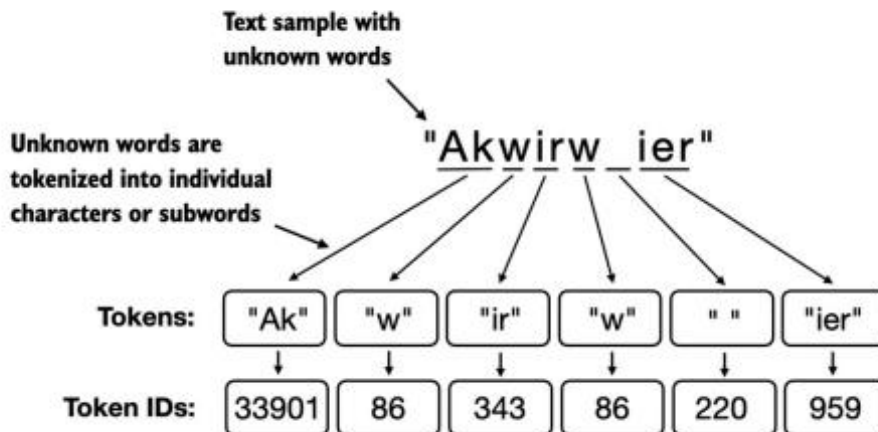
Komputer tidak memahami kata atau kalimat, melainkan angka. Langkah pertama adalah mengubah teks input menjadi unit-unit yang lebih kecil yang disebut token. Token bisa berupa kata utuh ("mobil"), sub-kata ("memper-", "-tanggungjawabkan"), atau bahkan karakter tunggal.

Contoh :

- Input : "LLM sangat kuat".

- Output (Tokens): ['LLM', 'sangat', 'kuat'] atau bisa juga ['L', 'L', 'M', 'sangat', 'ku', 'at'].

Setiap token ini kemudian dipetakan ke sebuah ID numerik unik dari kamus (vocabulary) yang dimiliki model seperti pada gambar 2.3.

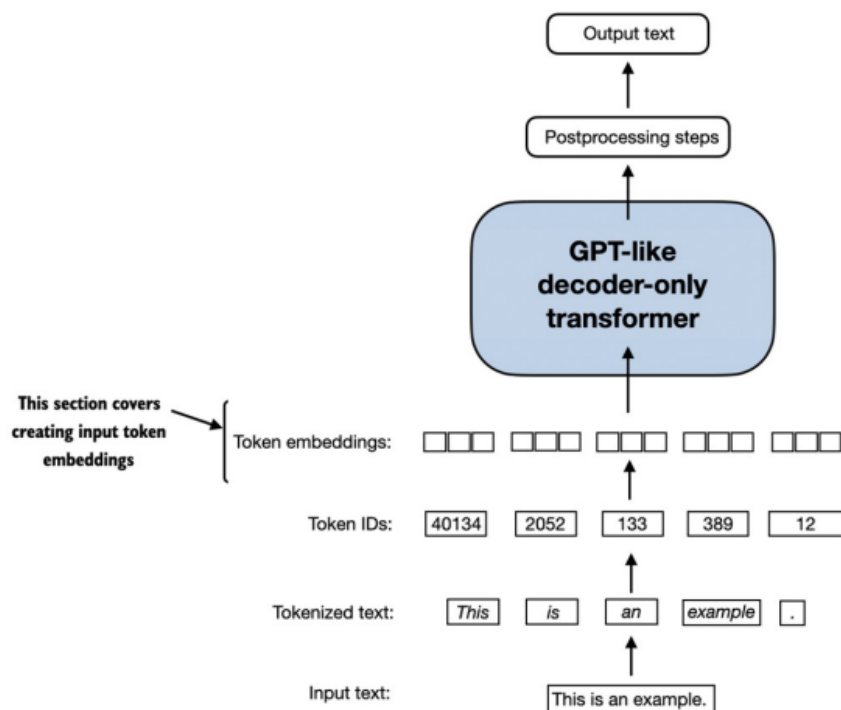


Gambar 2.3. Pemetaan Token Ke ID Numerik [12]

2. Embedding

Setiap token numerik kemudian diubah menjadi sebuah vektor atau sebuah daftar angka yang disebut embedding. Vektor ini bukanlah angka acak, ia merepresentasikan "makna" atau posisi semantik dari token tersebut dalam ruang konseptual multi-dimensi. Analogi: Bayangkan sebuah peta raksasa. Kata "raja" dan "ratu" akan berada berdekatan. Vektor dari ("raja" - "pria" + "wanita") akan menghasilkan vektor yang sangat dekat dengan "ratu". Embedding menangkap hubungan-hubungan semacam ini.

Secara praktis, proses transformasi dari teks mentah menjadi token, kemudian menjadi embedding yang kaya akan makna, dan akhirnya diproses oleh inti model LLM dapat divisualisasikan dalam alur kerja gambar 2.4 berikut, gambar ini mengilustrasikan skema kerja sebuah LLM berbasis arsitektur *decoder-only transformer*.



Gambar 2.4. Ilustrasi Skema Kerja LLM Berbasis *decoder-only transformer*[12]

3. Positional Encoding

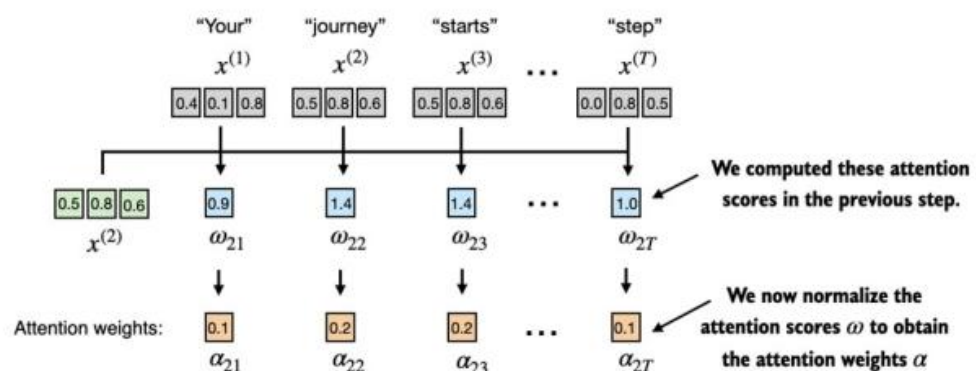
Salah satu kelemahan dasar dalam memproses kata sebagai daftar adalah hilangnya informasi urutan. Kalimat "Anjing menggigit pria" sangat berbeda dengan "Pria menggigit anjing". Untuk mengatasi ini, informasi posisi (urutan) dari setiap token ditambahkan ke vektor embedding-nya. Ini dilakukan menggunakan fungsi matematika (biasanya fungsi sinus dan kosinus) sehingga model tahu di mana setiap kata berada dalam kalimat.

4. Arsitektur Inti: The Transformer

Komponen ini merupakan inti dari arsitektur LLM modern. Arsitektur Transformer, yang diperkenalkan dalam paper "Attention Is All You Need" Vaswani dkk. [13] secara efisien memproses semua token dalam input secara simultan (paralel) dan mampu menentukan hubungan kontekstual di antara mereka.

Komponen kuncinya adalah Mekanisme Perhatian-Diri (*Self-Attention Mechanism*). Mekanisme ini bekerja dengan cara mengevaluasi semua kata lain dalam kalimat yang sama untuk setiap kata yang diproses. Tujuannya adalah untuk membangun pemahaman kontekstual dengan memberikan "skor perhatian" (*attention score*) yang lebih tinggi pada kata-kata yang paling relevan. Sebagai contoh, dalam kalimat "Robot itu mengambil bola karena ia terlalu berat", saat memproses token "ia", mekanisme *self-attention* akan memberikan skor perhatian tertinggi pada token "bola", bukan "robot", karena analisis gramatikal dan kontekstual mengidentifikasi "bola" sebagai referensi yang paling mungkin.

Mekanisme pemberian skor seperti pada contoh di atas melibatkan sebuah proses komputasi. Gambar 2.5 berikut menyajikan visualisasi sederhana tentang bagaimana skor perhatian dihitung dan dinormalisasi untuk setiap kata dalam sebuah kalimat.



Gambar 2.5. Visualisasi Attention Score Dan Normalisasi [12]

Kemampuan untuk menimbang relevansi secara dinamis inilah yang memungkinkan LLM memahami nuansa, mengatasi ambiguitas, dan menangani dependensi jangka panjang dalam teks. Hal ini merupakan sebuah kemajuan

signifikan dibandingkan arsitektur sekuensial sebelumnya seperti RNN dan LSTM, yang rentan terhadap hilangnya konteks awal (Hochreiter dkk.[14]).

5. Lapisan-Lapisan (Layers) dan Jaringan Feed-Forward

Output dari lapisan perhatian kemudian dilewatkan melalui jaringan saraf tiruan standar (*Feed-Forward Neural Network*) di setiap posisi token. Proses ini (*Attention* -> *Feed-Forward*) diulang berkali-kali dalam "lapisan-lapisan" (*layers*). LLM besar seperti GPT-3 memiliki 96 lapisan ini, memungkinkan mereka membangun pemahaman yang sangat kompleks dan abstrak dari teks.

6. Dekode dan Output (Decoding & Output Generation)

Setelah input diproses melalui semua lapisan Transformer, model menghasilkan representasi akhir yang kaya akan konteks. Dari sini, proses dekode dimulai:

1. Model menghasilkan distribusi probabilitas atas seluruh kosakata untuk token berikutnya.
2. Token dengan probabilitas tertinggi dipilih (atau dipilih secara acak berdasarkan distribusinya untuk variasi).
3. Token yang baru dihasilkan ini ditambahkan ke urutan input, dan seluruh proses diulangi lagi untuk menghasilkan token berikutnya, dan begitu seterusnya, hingga kalimat atau jawaban lengkap terbentuk.

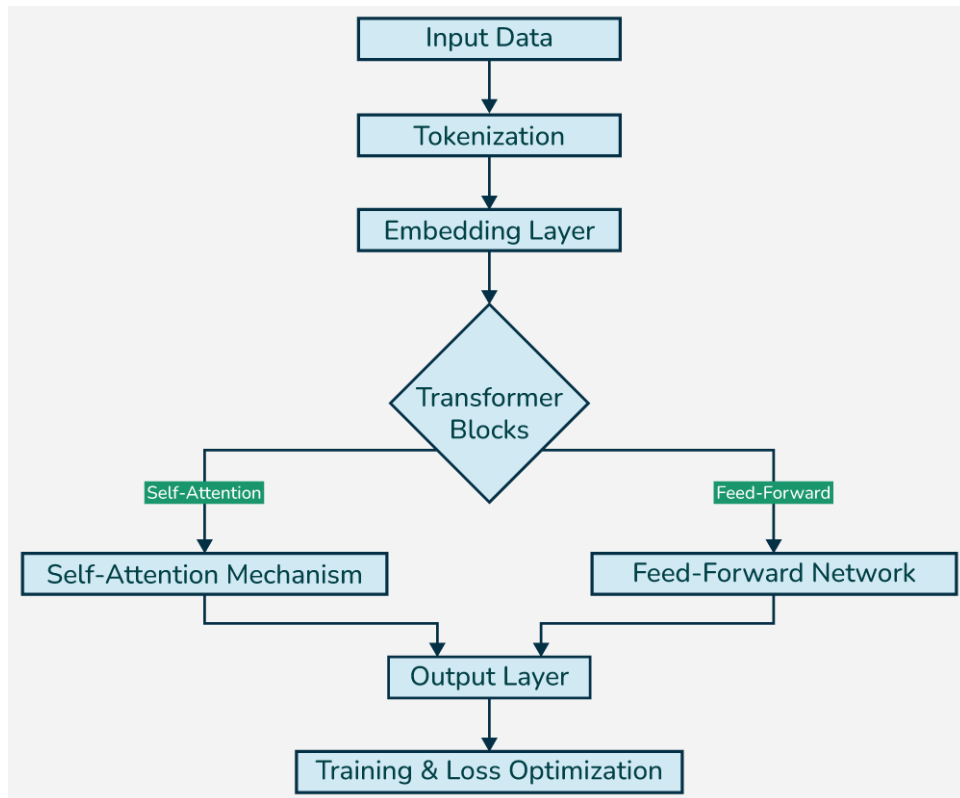
Setelah memahami komponen-komponen kunci seperti *embedding* dan mekanisme perhatian, penting untuk melihat bagaimana semua bagian ini terintegrasi dalam sebuah arsitektur yang utuh selama proses pelatihan model.

Proses kerja sebuah LLM berbasis Transformer dimulai dengan Input Data berupa teks mentah. Teks ini pertama-tama melalui tahap Tokenization, di mana

ia dipecah menjadi unit-unit yang lebih kecil (token). Selanjutnya, setiap token dikonversi menjadi representasi vektor numerik yang kaya akan makna melalui *embedding layer*.

Vektor *embedding* ini kemudian menjadi masukan bagi komponen inti, yaitu serangkaian *transformer blocks*. Seperti yang ditunjukkan dalam Gambar 2.6, setiap *transformer block* pada dasarnya terdiri atas dua subkomponen utama, yaitu *self-attention mechanism* yang bertugas menimbang relevansi dan membangun pemahaman kontekstual antar token, serta *feed-forward network* untuk pemrosesan lebih lanjut.

Hasil dari pemrosesan oleh blok-blok *transformer* ini kemudian dilewatkan ke *output layer* untuk menghasilkan keluaran awal. Dalam konteks pelatihan, seluruh alur ini diakhiri dengan proses *training & loss optimization*, di mana parameter model disesuaikan untuk meminimalkan kesalahan dan meningkatkan kinerjanya. Keseluruhan arsitektur dan alur data dari input hingga proses optimisasi ini dirangkum secara visual pada Gambar 2.6.



Gambar 2.6. Architecture of Large language model (LLMs)[15]

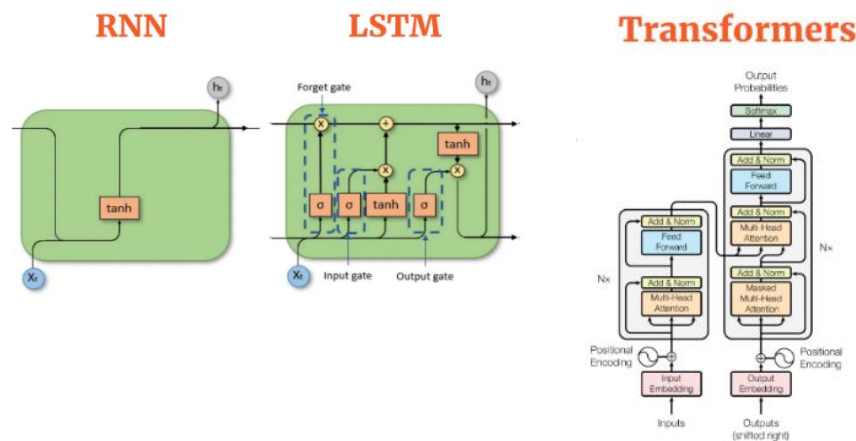
2.2.8.3. Arsitektur di Dalam LLM

Large Language Model (LLM) dibangun di atas beberapa jenis arsitektur fundamental. Arsitektur yang paling utama dan relevan dalam perkembangannya meliputi:

1. Transformer (Dominan): Seperti dijelaskan di atas, ini adalah arsitektur *de facto* untuk hampir semua LLM canggih saat ini. Keunggulannya adalah pemrosesan paralel dan mekanisme perhatian yang efektif menangani ketergantungan jarak jauh dalam teks.
2. RNN (Recurrent Neural Network) & LSTM (Long Short-Term Memory): Ini adalah arsitektur generasi sebelumnya. Mereka memproses data secara sekuensial (kata per kata). Meskipun revolusioner pada masanya, mereka memiliki dua kelemahan utama:

- Sulit diparalelkan: Karena sifatnya yang sekuensial, prosesnya lebih lambat.
- Vanishing Gradient Problem: Mereka kesulitan mengingat konteks dari kata-kata yang sangat jauh di awal kalimat atau paragraf (meskipun LSTM lebih baik dalam hal ini daripada RNN biasa)

Untuk memberikan gambaran yang lebih jelas mengenai perbedaan fundamental dalam alur pemrosesan data ini, Gambar 2.7 yang menyajikan perbandingan visual antara arsitektur sekuensial RNN , LSTM, GRU dan arsitektur paralel Transformer.



Gambar 2.7. Arsitektur Sekuensial RNN Dan Arsitektur Paralel Transformer[16]

2.2.8.4. Jenis-Jenis LLM

LLM dapat diklasifikasikan berdasarkan beberapa kriteria:

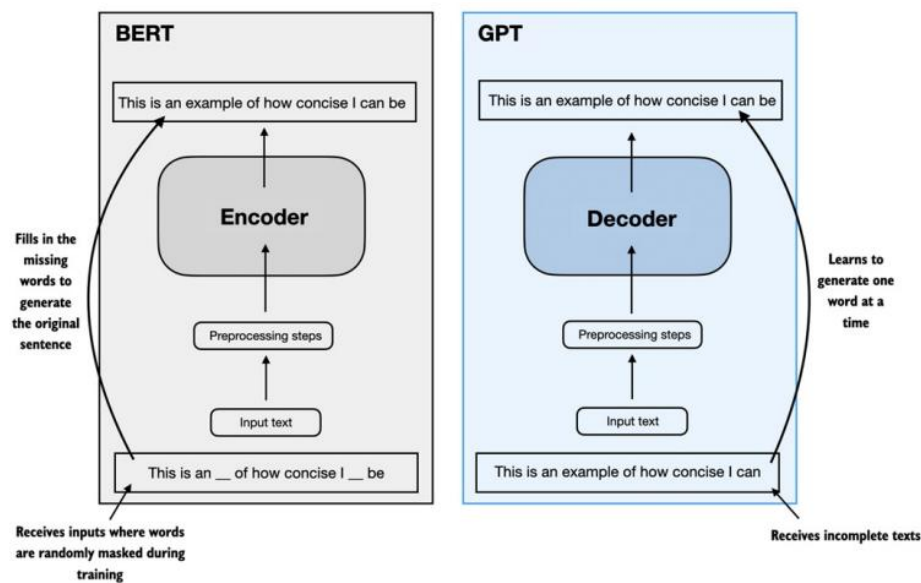
- Berdasarkan Arsitektur Transformer yang Digunakan:

1. Encoder-Only Models (Contoh: BERT, RoBERTa):

Model ini dirancang untuk memahami konteks secara mendalam (pemahaman dua arah/bidirectional). Mereka sangat baik untuk tugas-tugas analitis seperti klasifikasi teks, analisis sentimen, dan *named entity recognition*. Mereka tidak dirancang untuk menghasilkan teks panjang.

2. Decoder-Only Models (Contoh: GPT series, Llama, Gemini)

Model ini adalah apa yang kebanyakan orang kenal sebagai LLM. Mereka dilatih untuk memprediksi token berikutnya secara sekuensial (satu arah/unidirectional). Mereka unggul dalam tugas-tugas generatif: menulis esai, menjawab pertanyaan, *chatting*, dan peringkasan.



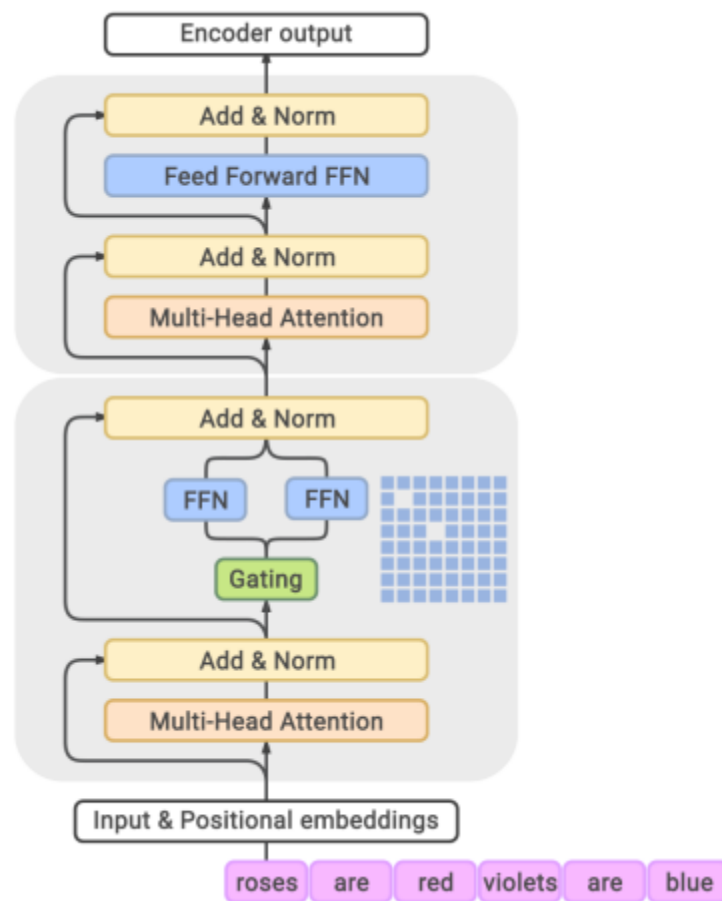
Gambar 2.8. Arsitektur Encoder-Only dan Decoder-Only.[12]

3. Arsitektur Lanjutan

Salah satu inovasi arsitektur yang paling signifikan untuk mencapai efisiensi pada skala model yang sangat besar adalah *Mixture-of-Experts* (MoE). Dalam arsitektur MoE, seperti yang diadopsi oleh Gemini 1.5 Pro, lapisan *Feed-Forward Network* (FFN) standar dalam sebuah blok Transformer digantikan oleh beberapa jaringan FFN paralel yang disebut "experts". Sebuah "gating network" yang ringan bertugas untuk secara dinamis memilih kombinasi "experts" mana yang paling relevan untuk diaktifkan untuk setiap token input. Dengan cara ini, model dapat memiliki jumlah parameter total

yang masif, namun biaya komputasi per token tetap rendah karena hanya sebagian kecil parameter yang digunakan.

Struktur internal dari sebuah blok Transformer yang telah dimodifikasi dengan MoE diilustrasikan pada Gambar 2.9. Diagram tersebut menunjukkan bagaimana setelah lapisan *Multi-Head Attention*, sebuah *Gating network* mengarahkan input ke beberapa FFN "expert" sebelum hasilnya digabungkan kembali.



Gambar 2.9. Struktur internal Transformer Dengan Modifikasi MoE [17]

- Berdasarkan Aksesibilitas:

1. Closed Source / API-based (Contoh: GPT-4 oleh OpenAI, Gemini oleh Google).
Model yang sangat canggih namun arsitektur dan bobot modelnya bersifat rahasia. Pengguna hanya bisa berinteraksi melalui via API saja.
2. Open Source (Contoh: Llama series oleh Meta, Mistral oleh Mistral AI, Falcon).
Model yang bobotnya dirilis secara publik. Ini memungkinkan peneliti dan pengembang untuk menjalankannya secara lokal, memodifikasinya (*fine-tuning*), dan membangun aplikasi di atasnya dengan lebih banyak kontrol.

2.2.8.5. Interaksi Dengan LLM

Keunggulan utama *Large Language Models* (LLM) terkini terletak pada fenomena *emergent behavior*, di mana model mampu menunjukkan keterampilan yang tidak diajarkan secara eksplisit melalui mekanisme *prompting* [12]. *Prompting* sendiri merupakan teknik pemberian instruksi tekstual untuk mengarahkan output LLM [18]. Terdapat tiga strategi utama dalam *prompting*:

- Zero-shot Learning: Model menyelesaikan tugas berdasarkan pengetahuan pra-pelatihannya tanpa contoh sebelumnya [12]. Misalnya, model dapat menerjemahkan "selamat pagi" menjadi "good morning" secara langsung.
- Few-shot Learning (In-Context Learning - ICL): Akurasi model ditingkatkan dengan menyertakan beberapa contoh input-output yang benar dalam *prompt* [12]. Pada model dengan konteks panjang seperti Gemini 1.5 Pro, metode ini bahkan memungkinkan penerjemahan bahasa langka seperti Kalamang hanya bermodalkan referensi tata bahasa dan kamus [19].
- Chain-of-Thought (CoT) Prompting: Untuk penyelesaian masalah kompleks, LLM diinstruksikan untuk menguraikan langkah penalaran ("berpikir langkah demi langkah") guna menghasilkan jawaban yang lebih presisi [18].

Strategi prompting yang digunakan adalah Few-shot Learning, di mana model diberikan serangkaian contoh input valid dan tidak valid (demonstrasi) untuk meningkatkan akurasi klasifikasi. Selain itu, prompt ini juga menerapkan Constraint-based Prompting dengan mendefinisikan aturan batasan (*validation rules*) secara eksplisit untuk memandu logika model. Contoh struktur prompt yang akan saya gunakan kurang lebih seperti berikut :

```
"{pre_validation_note}
Validate the input \"{type_str}\" from the following
{type_str}.
Check whether the input represents a valid company or
business name that could realistically exist.

Validation rules:
- The company name must not be random, meaningless, or dummy
text (e.g., 'asdf', 'qwerty', 'Lorem ipsum', 'Test Company').
- The name should contain recognizable business or corporate
elements that suggest it's a real company.
- The name may include common business suffixes like 'PT',
'CV', 'LLC', 'Inc.', 'Ltd.', 'Corp.', 'Co.', etc.
- The name length must be between 3 and 120 characters.
- The name cannot be a single generic word without business
context (e.g., 'Company', 'Business', 'Corporation').
- The name must not start or end with a space.
- Special characters like hyphens (-), periods (.),
ampersands (&), and numbers are acceptable if used
appropriately.
- Avoid clearly unnatural or random characters (e.g., #, $,
%, @) unless they are part of a legitimate company name.
- The name should sound professional and legitimate, not like
placeholder text.
- Company names may include industry-specific terms, founder
names, or descriptive business elements.

Valid company name examples:
- \"PT Telkom Indonesia Tbk\"
- \"Bank Central Asia\"
- \"Google LLC\"
- \"Microsoft Corporation\"
- \"Apple Inc.\"
- \"Amazon.com Inc.\"
- \"PT Bank Mandiri Tbk\"

Invalid company name examples:
- \"asdf company\"
- \"qwerty123\"
- \"Lorem Ipsum Corp\"

Important note:
Company names can vary significantly across different
jurisdictions and business structures. Accept both formal
corporate names (like 'PT Bank Mandiri Tbk') and informal
```

business names (like 'CV Sumber Makmur'). Company names may include founder names, industry terms, geographical references, or descriptive elements that help identify the business.

The key is that the name should appear to be a legitimate business entity that could realistically exist.

The output must be in the following JSON format (without any additional text):

```
{
  "valid": true or false,
  "message": "a short explanation of why it is valid or
invalid and use Bahasa Indonesia for this message response"
}
```

Input: "{input}"

Struktur prompt di atas secara spesifik mengatur mekanisme validasi nama perusahaan dengan menetapkan batasan logis dan contoh komparatif untuk menyaring input teks yang tidak wajar atau acak. Dengan memaksakan format keluaran JSON yang baku serta penjelasan dalam Bahasa Indonesia, pendekatan ini memastikan sistem memperoleh hasil verifikasi yang akurat, terstruktur, dan siap diintegrasikan secara otomatis.